

January 2, 2026

Contents

1	Introduction	2
2	Part 1: SQLite Database	3
2.1	Purpose	3
2.2	Schema	3
2.3	Sample Data	3
2.4	Creation	4
3	Part 2: Jupyter Notebook Analysis	4
3.1	Overview	4
3.2	Cell Structure	4
3.3	Key Libraries	4
4	Part 3: Machine Learning Model	5
4.1	Algorithm	5
4.2	How It Works	5
4.3	Implementation	5
4.4	Strengths	5
4.5	Limitations	5
5	Part 4: Generative AI Integration	6
5.1	OpenAI API	6
5.2	Process	6
5.3	Implementation	6
5.4	Response Parsing	6
5.5	API Considerations	6
6	Part 5: Data Visualization	6
6.1	Plotly Dashboard	6
6.2	Chart Types	7
6.3	Components	7
6.4	Features	7

7	Part 6: FastAPI REST API	7
7.1	Overview	7
7.2	Endpoints	7
7.3	Implementation	8
7.4	Running	8
7.5	Usage	8
8	Part 7: Docker Containerization	8
8.1	Purpose	8
8.2	Dockerfile	8
8.3	Building	8
8.4	Running	9
8.5	Benefits	9
9	Part 8: Kubernetes Orchestration	9
9.1	Overview	9
9.2	Deployment Configuration	9
9.3	Service Configuration	9
9.4	Deployment Commands	10
9.5	Features	10
10	Part 9: Jenkins CI/CD Pipeline	10
10.1	Purpose	10
10.2	Jenkinsfile	10
10.3	Pipeline Stages	11
10.4	Triggering	11
10.5	Benefits	11
11	System Architecture	11
11.1	Data Flow	11
11.2	Integration Points	12
12	Getting Started	12
12.1	Prerequisites	12
12.2	Installation Steps	12
13	Troubleshooting	13
13.1	Common Issues	13
13.2	Debugging	13
14	Summary	13

1 Introduction

This project demonstrates a complete modern data science and DevOps architecture combining multiple technologies:

- Database management with SQLite
- Interactive analysis with Jupyter Notebook
- Machine learning predictions
- Generative AI integration
- Data visualization
- REST API development
- Container orchestration
- Continuous integration and deployment

2 Part 1: SQLite Database

2.1 Purpose

The SQLite database stores all historical sales data. It serves as the single source of truth for our forecasting system.

2.2 Schema

The sales table contains these columns:

- id - Primary key, auto-incremented record identifier
- product - Product name (Widget A, Widget B, etc.)
- region - Geographic region (North, South, East, West)
- date - Transaction date in YYYY-MM-DD format
- units_sold - Integer number of units sold
- revenue - Decimal revenue amount

2.3 Sample Data

Example records from the sales table:

id	product	region	date	units_sold	revenue
1	Widget A	North	2025-01-01	10	100.00
2	Widget B	South	2025-01-01	5	50.00
3	Widget A	North	2025-01-02	12	120.00

2.4 Creation

Create the database using sales.sql:

```
sqlite3 sales.db < sales.sql
```

3 Part 2: Jupyter Notebook Analysis

3.1 Overview

The Jupyter notebook contains 10 interconnected cells that form the analysis pipeline.

3.2 Cell Structure

Cell 1: Data Loading and Export

Connects to SQLite, loads all sales data, exports to CSV for backup, filters by region.

Cell 2-3: Package Installation

Installs Plotly, Dash, and other required libraries for visualization and web framework.

Cell 4: OpenAI API Integration

Calls OpenAI's ChatGPT model with historical sales data to generate AI predictions.

Cell 5: Response Parsing

Extracts numerical predictions from AI text response using regex parsing.

Cell 6: Actual Sales Visualization

Creates interactive Plotly line chart showing historical sales trends.

Cell 7: Predicted Sales Visualization

Creates interactive Plotly chart showing forecast predictions.

Cell 8: FastAPI Endpoint Definition

Defines REST API endpoints for /sales and /predict routes.

Cell 9: API Testing

Tests endpoints and validates responses with error handling.

Cell 10: Server Launch

Starts the FastAPI server with uvicorn for local testing.

3.3 Key Libraries

pandas - Data manipulation and analysis

sqlite3 - Database connectivity

plotly - Interactive visualizations

scikit-learn - Machine learning algorithms

openai - OpenAI API client

fastapi - Modern web framework

uvicorn - ASGI server

4 Part 3: Machine Learning Model

4.1 Algorithm

LinearRegression from scikit-learn provides baseline forecasting by learning linear trends.

4.2 How It Works

The model fits a line through historical data points and extrapolates to future dates.

Example: If sales increased by 2 units per day, the model predicts this trend continues.

4.3 Implementation

```
from sklearn.linear_model import LinearRegression
import pandas as pd

# Convert dates to ordinal numbers
X = data['date_ordinal'].values.reshape(-1, 1)
y = data['units_sold'].values

# Train model
model = LinearRegression()
model.fit(X, y)

# Predict next 7 days
future_dates = [[date1], [date2], ..., [date7]]
predictions = model.predict(future_dates)
```

4.4 Strengths

Simple and interpretable

- Fast training and prediction
- Low computational overhead
- Works with limited data

4.5 Limitations

Cannot capture complex nonlinear patterns

- Assumes constant trend
- Sensitive to outliers
- No seasonal adjustment

5 Part 4: Generative AI Integration

5.1 OpenAI API

Uses ChatGPT (gpt-4o-mini model) for intelligent sales analysis.

5.2 Process

1. Format historical sales data as text 2. Send prompt to OpenAI API 3. Receive AI-generated forecast 4. Parse numbers from response 5. Use as additional prediction source

5.3 Implementation

```
from openai import OpenAI

client = OpenAI(api_key='YOUR_KEY')

history = data.to_dict('records')
prompt = f'Predict sales: {history}'

response = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[{'role': 'user', 'content': prompt}]
)

prediction = response.choices[0].message.content
```

5.4 Response Parsing

Uses regex to extract numbers from AI text response:

```
import re
numbers = re.findall(r'\b\d+\b', prediction)
forecast = [int(n) for n in numbers[:7]]
```

5.5 API Considerations

Requires valid API key from platform.openai.com
Incurs cost per API call
gpt-4o-mini offers good balance of speed and accuracy

6 Part 5: Data Visualization

6.1 Plotly Dashboard

Interactive charts for exploring sales data and predictions.

6.2 Chart Types

- Line charts for time-series trends
- Scatter plots with markers for predictions
- Interactive legends and hover tooltips
- Responsive sizing for different displays

6.3 Components

```
import plotly.graph_objects as go

# Actual sales chart
fig1 = go.Figure()
fig1.add_trace(go.Scatter(
    x=data['date'],
    y=data['units_sold'],
    mode='lines',
    name='Actual Sales'
))

# Predicted sales chart
fig2 = go.Figure()
fig2.add_trace(go.Scatter(
    x=future_dates,
    y=predictions,
    mode='lines+markers',
    name='Forecast'
))
```

6.4 Features

- Hover information shows exact values
- Click legend items to toggle visibility
- Download charts as PNG
- Zoom and pan capabilities

7 Part 6: FastAPI REST API

7.1 Overview

Exposes predictions via HTTP endpoints for external applications.

7.2 Endpoints

- GET /sales - Returns all sales records as JSON
- GET /predict - Returns forecast for specified region

GET /docs - Interactive API documentation (Swagger UI)

7.3 Implementation

```
from fastapi import FastAPI

app = FastAPI()

@app.get('/sales')
def get_sales():
    return data.to_dict('records')

@app.get('/predict')
def predict(region: str = 'North'):
    return {'region': region, 'forecast': predictions}
```

7.4 Running

```
uvicorn api:app --host 0.0.0.0 --port 8000
```

Access at <http://localhost:8000/docs>

7.5 Usage

```
curl http://localhost:8000/sales
curl http://localhost:8000/predict?region=North
```

8 Part 7: Docker Containerization

8.1 Purpose

Packages application with all dependencies in isolated container.

8.2 Dockerfile

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

8.3 Building

```
docker build -t sales-app:latest .
```


8.4 Running

```
docker run -d -p 8000:8000 sales-app:latest
docker logs <container-id>
docker stop <container-id>
```

8.5 Benefits

- Consistent environment across machines
- Simplified deployment process
- Easy version management
- Isolation from host system

9 Part 8: Kubernetes Orchestration

9.1 Overview

Manages containerized applications at scale across multiple servers.

9.2 Deployment Configuration

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sales-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: sales-app
  template:
    metadata:
      labels:
        app: sales-app
    spec:
      containers:
        - name: sales-app
          image: sales-app:latest
          ports:
            - containerPort: 8000
```

9.3 Service Configuration

```
apiVersion: v1
kind: Service
metadata:
```

```

    name: sales-service
spec:
  type: LoadBalancer
  selector:
    app: sales-app
  ports:
  - port: 80
    targetPort: 8000

```

9.4 Deployment Commands

```

kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get pods
kubectl get services
kubectl logs <pod-name>

```

9.5 Features

- Automatic pod restart on failure
- Load balancing across replicas
- Rolling updates with zero downtime
- Horizontal scaling

10 Part 9: Jenkins CI/CD Pipeline

10.1 Purpose

Automates testing, building, and deployment when code changes.

10.2 Jenkinsfile

```

pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        git url: 'https://github.com/user/repo.git'
      }
    }

    stage('Build') {
      steps {
        sh 'docker build -t sales-app .'
      }
    }
  }
}

```

```

    }

    stage('Test') {
        steps {
            sh 'python -m pytest tests/'
        }
    }

    stage('Deploy') {
        steps {
            sh 'kubectl apply -f deployment.yaml'
        }
    }
}

```

10.3 Pipeline Stages

Checkout - Pull code from GitHub
 Build - Create Docker image
 Test - Run unit tests
 Deploy - Update Kubernetes deployment

10.4 Triggering

Webhook from GitHub on push
 Scheduled builds at specific times
 Manual trigger via Jenkins UI

10.5 Benefits

Automated testing catches errors early
 Consistent build process
 Fast deployment cycles
 Complete deployment history

11 System Architecture

11.1 Data Flow

SQLite Database stores historical sales data
 Jupyter loads data and generates analysis
 ML model learns patterns from historical data
 OpenAI API provides intelligent predictions
 Plotly creates interactive visualizations
 FastAPI serves predictions via HTTP

Docker containers package the application
Kubernetes orchestrates container deployment
Jenkins automates the entire pipeline

11.2 Integration Points

Database provides input data
ML and AI produce predictions
Visualization and API consume predictions
Containers enable portability
Kubernetes provides scalability
Jenkins automates deployment

12 Getting Started

12.1 Prerequisites

Python 3.9 or higher
pip package manager
Docker installed
kubectl configured for Kubernetes cluster
Jenkins server running

12.2 Installation Steps

1. Clone repository from GitHub
2. Install Python dependencies:

```
pip install -r requirements.txt
```

3. Create SQLite database:

```
sqlite3 sales.db < sales.sql
```

4. Set OpenAI API key:

```
export OPENAI_API_KEY='your-key-here'
```

5. Launch Jupyter notebook:

```
jupyter notebook sales.ipynb
```

6. Execute cells in order from Cell 1 to Cell 10
7. Access API documentation at <http://localhost:8000/docs>

13 Troubleshooting

13.1 Common Issues

Database connection failed - Ensure sales.db exists and has correct permissions
API key invalid - Verify OpenAI key is correct and has remaining credits
Port already in use - Kill existing process or use different port
Docker build fails - Check Docker daemon is running and Dockerfile is valid
Kubernetes pod crashes - Check logs with `kubectl logs` command

13.2 Debugging

View database contents: `sqlite3 sales.db`
Check API logs: `docker logs container-name`
Inspect pod status: `kubectl describe pod pod-name`
Monitor resource usage: `kubectl top nodes`

14 Summary

This integrated system demonstrates:

- Data persistence with SQLite
- Interactive analysis with Jupyter
- Machine learning predictions
- Generative AI integration
- Data visualization with Plotly
- Web API with FastAPI
- Container management with Docker
- Orchestration with Kubernetes
- Automation with Jenkins

All components work together to create a production-ready forecasting platform capable of analyzing historical data, generating predictions, and serving results via scalable cloud infrastructure.